itthynk

SOFTWARE DEVELOPMENT INTERNSHIP PROGRAMME · COHORT 2026

Demo moved · now Monday 14 Sep, not Friday

Inventory Management System — Week 4 Feedback

What exists — the login endpoint, the cookie handling, the CORS setup — is genuinely built correctly. The lesson worth focusing on is what was chosen to build first: a lot of visible surface (pages, layouts, one login screen) went in before the one thing everything else depends on — a real way for more than one person to have an account.

Repo · `cohort-2026/inventory-management-system`

Sprint work · Authentication & Authorization, Mon 7 – Fri 11 Sep 2026 **Demo** · Monday 14 Sep 2026 (moved from Friday)

WHAT THIS DOCUMENT IS — AND ISN'T

This is **feedback, not a fix-it list.** Every item below is here to help you see where your own reasoning went wrong and think through a better way to approach it — not to hand you code to paste in. Each one points at the real file and line in your repo so you can go look at it yourselves, and some show the actual code that's there right now — not because copying a replacement is the point, but because seeing your own code named directly is more useful than a description of it. The short examples further down are illustrative, to make an idea concrete, not to be copied.

What to demo on Monday

The brief's own sequence, walked through against what this repo actually does today — from the same audit this feedback is drawn from.

Before you present: `testing` was merged into `main` this morning (PR #77) — you're clear to present from `main` now. The gaps below are exactly the same as before the merge, just on the branch you'd normally check out.

Step 1 Register a new account

Not built

ON SCREEN

Nothing — there's no sign-up form.

IN THE CODE / TERMINAL

No POST `/api/register` or equivalent exists on any branch, confirmed across the full git history.

This needs to be built before Monday, not worked around — see Solution 1.

Step 2 Log in

One account only

ON SCREEN

Works, but only with the one hardcoded "admin" account.

IN THE CODE / TERMINAL

`Set-Cookie` is present and httpOnly — good — but missing `secure` and `sameSite` flags (`login.ts:27-30`).

Say out loud in the demo that this is the one seeded account, not a real login system yet. See Solution 2 for the cookie flags.

Step 3 Browse the inventory / product pages

Not real data

ON SCREEN

Data displays and looks complete.

IN THE CODE / TERMINAL

It's entirely mock data — e.g. `ProductList.tsx:19` comments "swap for a fetch to GET /api/products once that route exists."

Say this plainly in the demo rather than presenting it as live — it will come up in questions either way.

Step 4 Try an admin-only action as a non-admin user

Not built

ON SCREEN

N/A — there is no second account to log in as.

IN THE CODE / TERMINAL

No `requireRole` middleware exists anywhere yet.

See Solution 3.

Step 5 Log out

Ready

ON SCREEN

Works cleanly, returns to login.

IN THE CODE / TERMINAL

`res.clearCookie('token', ...)` — genuinely correct, one of the few complete pieces.

Step 6 Show CORS + cookies working across ports

Ready — highlight this

ON SCREEN

N/A — check the Network tab.

IN THE CODE / TERMINAL

`cors({ origin: 'http://localhost:5173', credentials: true })` on the backend, `credentials: 'include'` on every frontend call — both sides correct.

This is genuinely one of only two teams that got this fully right end-to-end. Lead with it.

RECOMMENDED ORDER TO WORK THROUGH THIS

WHEN	ISSUE	WHERE
1st	Build the real users table and registration endpoint — everything else depends on this	Lesson 1
2nd	Add a role-checking middleware once a second account can exist	Lesson 4
3rd	Add the missing cookie flags and remove the hardcoded JWT fallback	Lesson 3
anytime	Show the real server error on the login screen, and label the mock inventory data clearly	Lessons 5 & 2

Registration is the one thing everything else is downstream of — roles, a second demo account, real ownership checks all need it to exist first.

Where this went wrong, and how to think about it differently

In the order it'll matter most — not necessarily the order you'll want to tackle it.

1 One account was built instead of a login system

`backend/src/routes/login.ts:8-11`

WHAT WENT WRONG

> Login currently checks a single, hardcoded username and password written directly into the code. It works, in the sense that it lets one specific person in — but there's no way for anyone else to ever have an account, because there's no registration, and no table anywhere that a second account could even live in.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
const FAKE_USER = {
    username: 'admin',
    password: '$2b$10$Aj37q1FLEdFvD60I3v2OJeWI/vVCq6cTBnLvGLJmlNAJWatGT4MmG'
}
```

WHY IT MATTERS

> *It's the difference between building a house with one specific key that was cut before the house existed, versus building a house with a lock that can be given new keys. The first one lets exactly one person in, forever. It isn't a smaller version of a real lock — it's a different thing altogether.*

A BETTER WAY TO THINK ABOUT IT

> Start from "how does a second person get an account" rather than "how does this one person get in." A real users table and a registration endpoint are what make everything that follows — roles, permissions, a second demo account — even possible to build. Right now, those are blocked, not just unfinished.

EXAMPLE

illustrative only

```
CREATE TABLE users (
  id            SERIAL PRIMARY KEY,
  email         TEXT NOT NULL UNIQUE,
  password_hash TEXT NOT NULL,
  role          TEXT NOT NULL DEFAULT 'staff'
);
// once this exists, login.ts can look someone up instead of comparing
// against one name and password written into the file itself
```

2 The shop window was built before the stockroom

`frontend/src/pages/ProductList.tsx:19 (same pattern in InventoryOverview.tsx, ProductDetails.tsx, StockMovementHistory.tsx)`

WHAT WENT WRONG

> The inventory pages show real-looking product data, complete with layout and detail — but none of it comes from the server. It's written directly into the frontend as placeholder values, with a comment noting it'll be connected "once that route exists."

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
// Mock data — swap for a fetch to GET /api/products once that route exists.
```

WHY IT MATTERS

> *It's like dressing a shop window with photographs of products instead of the actual stock — from the street it looks fully open for business. Walk through the door, and there's nothing in the stockroom to actually sell.*

A BETTER WAY TO THINK ABOUT IT

> It's a completely reasonable way to work out what a screen should look like early — but it's worth treating that placeholder data as a visible flag, not something to quietly leave in place. A short comment or a banner saying "sample data" costs nothing and means nobody mistakes the display for the real thing later, including during a demo.

3 A backup plan that was never meant to be used, but quietly could be

`backend/src/routes/login.ts:25`

WHAT WENT WRONG

> If the app's secret signing key isn't set, the code falls back to a fixed, hardcoded value instead of refusing to start. Locally, that's invisible — everything just works. But that same fallback value sits in the code for anyone to find, and if it were ever relied on somewhere real, it would mean every session token could be forged by someone who simply read the source.

WHAT'S ACTUALLY THERE RIGHT NOW

The audit recorded a hardcoded `'dev-secret'` fallback used when `JWT_SECRET` is unset, at this line. Missing `secure`/`sameSite` flags on the session cookie were found in the same file, lines 27–30.

WHY IT MATTERS

> *It's like a spare house key kept under a rock "just in case the real one doesn't work" — except the rock's location is written on a public noticeboard. It's only ever meant to be a fallback. But a fallback that's this easy to find isn't really a fallback at all.*

A BETTER WAY TO THINK ABOUT IT

> When something is genuinely required — like a secret key — it's often safer for the program to refuse to start at all if it's missing, rather than quietly substituting something that happens to work today. A loud failure during setup is far easier to catch than a silent one that only matters months later.

EXAMPLE

illustrative only

```
// instead of quietly falling back to a default:
const JWT_SECRET = process.env.JWT_SECRET;
if (!JWT_SECRET) throw new Error("JWT_SECRET is not set — refusing to start");
```

4 There's a lock, but nothing yet says who's allowed to use it

`no requireRole middleware anywhere in the repo · no protected business route exists yet`

WHAT WENT WRONG

> The one piece that checks "is someone signed in" (requireAuth) works correctly — but there's no piece anywhere that checks "is this specific person allowed to do this specific thing". Since there's also only one account in the whole system, this hasn't been possible to build or test yet either way.

WHAT'S ACTUALLY THERE RIGHT NOW

`requireAuth` is applied to exactly one route (`GET /api/me`). There is no role-checking middleware anywhere, and no other backend route exists yet for it to protect.

WHY IT MATTERS

> *It's like a building with a working front-door badge reader, but no record anywhere of which badges are allowed into which rooms — right now it only matters because there's exactly one badge in existence.*

A BETTER WAY TO THINK ABOUT IT

> Once a second real account can exist (see the first lesson above), the next real step is a role check most routes can share — same shape as the sign-in check, but answering "what am I allowed to do" instead of just "who am I".

EXAMPLE

illustrative only

```
export function requireRole(...allowed) {
  return (req, res, next) => {
    if (!allowed.includes(req.user?.role)) {
      return res.status(403).json({ error: "Not allowed" });
    }
    next();
  };
}
// usage: router.delete("/api/products/:id", requireAuth, requireRole("admin"),
handler)
```

5 The error message on screen doesn't say what actually went wrong

`frontend/src/pages/Login.tsx:24`

WHAT WENT WRONG

> When login fails, the screen always shows the same generic 'Login failed' message — even though the server sends back a more specific reason. The more useful message is being thrown away rather than shown.

WHAT'S ACTUALLY THERE RIGHT NOW

from the audit, quoted from the repo, not paraphrased

```
catch (err) {
  setError("Login failed"); // the real message from the server is discarded here
}
```

WHY IT MATTERS

> *It's like a phone system that always says 'call failed' whether the number was busy, disconnected, or simply mistyped — technically true every time, but useless for knowing what to actually do next.*

A BETTER WAY TO THINK ABOUT IT

> When a server already sends back a clear reason for a failure, showing that reason (rather than a fixed generic string) usually costs nothing extra to build and saves real confusion later — for users and for whoever's debugging it.

Checklist — work through this before Monday

One box per lesson above, phrased as something you can actually check off. Tick it once you can point to where it's true in your own code — not once you've read the explanation. Items marked **Critical** are where the audit found a real gap; be ready to show it live, not describe it.

- [ ] **Can a second person register their own account, independent of the one hardcoded login?** CRITICAL
- [ ] Is the mock inventory data clearly labelled as sample data, wherever it's shown?
- [ ] **Does the app refuse to start if JWT_SECRET is missing, instead of silently using a default?** CRITICAL
- [ ] **Does a requireRole (or equivalent) middleware exist, gating at least one real route?** CRITICAL
- [ ] Does the login screen show the server's actual error message instead of a fixed generic one?